PQC: Lattice-Based Cryptography

Paul Joo-Hyun Kim (pauljoohyunkim@gmail.com)

November 2, 2026

Post-Quantum Cryptography

Need for Post-Quantum Cryptography

- Quantum algorithms (e.g. Shor's algorithm) can attack RSA, ECC, etc.
  - Polynomial-time attack
  - $O\left((\log N)^3\right)$

Types of PQC

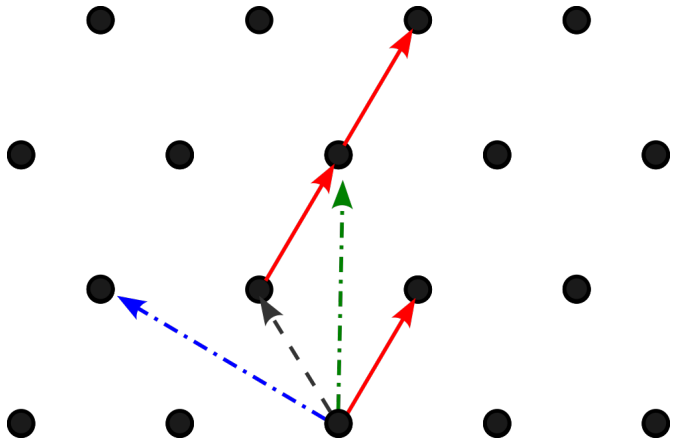
Algorithms

- **Lattice**
- Multivariate
- Hash
- Code
- Isogeny

In this presentation...

We will focus on ML-KEM (AKA CRYSTALS-Kyber), a key ~~exchange algorithm~~ encapsulation mechanism (KEM).

Lattice



Lattice

A lattice Λ in $\mathbb{R}^n$ has the form:

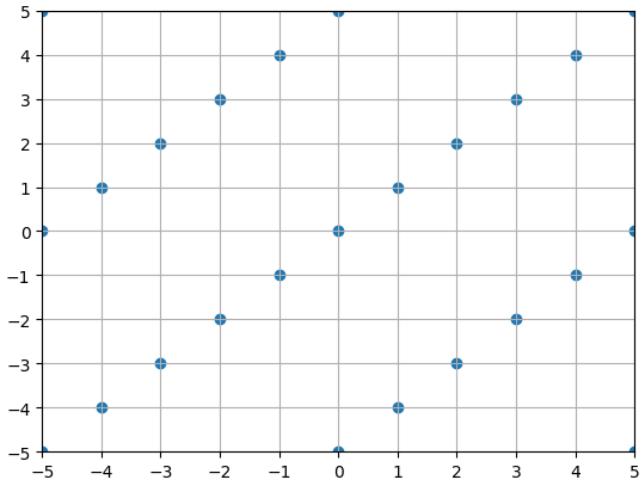
$$\Lambda = \left\{ \sum_{i=1}^{n} a_i v_i \middle| a_i \in \mathbb{Z} \right\}$$

where the set of v_i form basis.

Example

Using bases

$$\mathcal{B} = \left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} -2 \\ 3 \end{pmatrix} \right\}$$



Polynomial Ring

Ring

Ring R defines a set with addition and multiplication. **Division is not necessarily defined**.
Examples: Integers ($\mathbb{Z}$), modulo n ($\mathbb{Z}_n$), etc.

Polynomial Ring

In a nutshell, polynomial ring $R[x]$ is a set of polynomials from a ring R.

Ideal

Ideal

An ideal I of ring R is a subset such that is an additive subgroup and has absorption property.

$$x \in I, r \in R \Longrightarrow rx \in I, xr \in I$$

Examples

- $n\mathbb{Z} = \{0, \pm n, \pm 2n, \cdots\}$: multiples of n inside the ring $\mathbb{Z}$.
- $(x^n + 1)\mathbb{R}[x] = \{(x^n + 1)p(x) | p(x) \in \mathbb{R}[x]\}$ in the ring $\mathbb{R}[x]$.

Quotient Ring

Given ideal I and ring R, the set of all cosets

$$R/I = \{r + I | r \in R\}$$

is a quotient ring.

Example: Modulo

$\mathbb{Z}_n = \{0, 1, 2, \cdots, n-1\}$ in modulo n is actually

$$\mathbb{Z}_n = \mathbb{Z}/n\mathbb{Z} = \{0 + n\mathbb{Z}, 1 + n\mathbb{Z}, \cdots, (n-1) + n\mathbb{Z}\}$$

Quotient Ring in ML-KEM

ML-KEM Quotient Ring

The ring of interest in **ML-KEM** is

$$R_q = \mathbb{Z}_q[x]/\left(x^{256}+1\right)$$

where $q = 3329$.
Note that $|R_q| = 3329^{256} \approx 5.156 \times 10^{901}$.
Compare that to key size of AES-256 ($\approx 1.16 \times 10^{77}$)

Example

- $1 + 329x + 2840x^2 + 3328x^3 + \cdots + 293x^{255} \in R_q$
- $x + 326x^{256} \equiv x - 326$

Module-LWE

Learning With Errors (LWE)

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$$

where

- $\mathbf{b} \in R_q^{3\times 1}$ is public key.
- $\mathbf{A} \in R_q^{3\times 3}$ is public matrix.
- $\mathbf{s} \in R_q^{3\times 1}$ is private key.
- $\mathbf{e} \in R_q^{3\times 1}$ is "noise" polynomial, or "error".

Learning With Errors (LWE)

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$$

Note that $\mathbf{A}$ is a 3×3 matrix of polynomial of degree 255, so there are $3 \times 3 \times 256 = 2304$ variables.

Error

The error term $\mathbf{e}$ provides security.
Without e, this is a matrix equation that can be easily solved by Gaussian elimination ($O\left(\frac{2}{3}n^3\right)$)

ML-KEM (Kyber-768)

Agreeing on Secret (Setup)

Goal: Alice wants to send a 256-bit AES key to Bob.

- Setup
  1. Bob generates $\mathbf{s}$ and $\mathbf{e}$.
  2. Bob picks a random public matrix $\mathbf{A}$.
     - In practice, a seed is sent, which is used with SHAKE128 to generate this.
  3. Bob computes $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$
  4. Bob sends $\mathbf{A}$ and $\mathbf{b}$ to Alice.

Agreeing on Secret (Encapsulation)

- Encapsulation
  1. Alice generates $\mathbf{r}$ a secret vector.
  2. $\mathbf{e}_1$, a nosie vector.
  3. e_2, a noise polynomial.
  4. m, the 256-bit message (secret AES key) encoded as polynomial.
     - $01001\cdots \Longrightarrow 0 + x + 0x^2 + 0x^3 + x^4 + \cdots$
  5. Compute lattice piece $\mathbf{u} = \mathbf{A}^T\mathbf{r} + \mathbf{e}_1$.
  6. Compute masking piece $v = \mathbf{b}^T\mathbf{r} + e_2 + \text{Decompress}(m)$
     - "Decompress" means to map coefficients of m in a way that $0 \mapsto 0$ and $1 \mapsto \approx q/2 = 1664$.
  7. Alice sends $(\mathbf{u}, v)$

Agreeing on Secret (Decapsulation)

- Decapsulation
  1. Bob computes:

$$\begin{aligned} v - \mathbf{s}^T\mathbf{u} &= \mathbf{b}^T\mathbf{r} + e_2 + \text{Decompress}(m) - \mathbf{s}^T\left(\mathbf{A}^T\mathbf{r} + \mathbf{e}_1\right) \\ &= \left(\mathbf{As}\right)^T\mathbf{r} - \mathbf{s}^T\left(\mathbf{A}^T\mathbf{r}\right) + \text{Decompress}(m) + \epsilon \\ &= \text{Decompress}(m) + \epsilon \end{aligned}$$

     where ϵ is a small error term.
  2. To get m, round coefficient.
     - If a coefficient is closer to $q/2$, then map it to 1.
     - If a coefficient is closer to 0, then map it to 0.

Number Theoretic Transform (NTT)

Optimizing Polynomial Multiplication

Traditional multiplication of polynomial $u, v \in R_q$:

$$\begin{pmatrix} u_0 & u_1 & u_2 & u_3 & \cdots & u_{254} & u_{255} \end{pmatrix} \begin{pmatrix} v_0 & v_1 & \cdots & v_{255} \\ -v_{255} & v_0 & \cdots & v_{254} \\ -v_{254} & -v_{255} & \cdots & v_{253} \\ \vdots & \vdots & \ddots & \vdots \\ -v_1 & -v_2 & \cdots & v_0 \end{pmatrix}$$

takes $O(N^2)$ where $N = 256$ in this case due to $x^N + 1$.

Optimizing Polynomial Multiplication

There is map NTT to convert from $f \in R_q$ to $\hat{f} \in \mathbb{Z}_q^{256}$ while preserving algebraic structure such that:

$$\text{NTT}(f \times g) = \hat{f} \odot \hat{g}$$

where is a "special" product $\hat{f} \odot \hat{g}$ takes $O(n)$.
It also takes $O(n \log n)$ operations for NTT, so overall, this method is faster.

Incomplete NTT

In ML-KEM, a variant of NTT called the **incomplete NTT** is used, as $x^{256} + 1$ does not have 256th root.

Incomplete NTT

Note that $17^{128} \equiv -1 \mod q$. Call $\zeta = 17$.

$$
\begin{aligned}
x^{256} + 1 &= x^{256} - \zeta^{128} \\
&= \left(x^{128} + \zeta^{64}\right)\left(x^{128} - \zeta^{64}\right) \\
&= \left(x^{64} - \zeta^{64+128}\right)\left(x^{128} - \zeta^{64}\right) \\
&= \left(x^{32} + \zeta^{32+64}\right)\left(x^{32} - \zeta^{32+64}\right)\left(x^{64} + \zeta^{32}\right)\left(x^{64} - \zeta^{32}\right) \\
&= \vdots \\
&= \prod_{i=0}^{127}\left(x^2 - \zeta^{2\mathsf{BitRev}_7(i)+1}\right)
\end{aligned}
$$

Incomplete NTT (cont'd)

Hence for $f \in R_q$,

$$\begin{aligned}
&\text{NTT}(f) \\
&= \text{PROJ}\left(\left(\left(\underbrace{f \mod \left(x^2 - \zeta^{2\text{BitRev}_7(i)+1}\right)}_{\hat{f}[2i]+\hat{f}[2i+1]x} | i \in \{0, 1, \cdots, 127\}\right)\right)\right) \\
&= \left(\hat{f}[0], \hat{f}[1], \cdots, \hat{f}[255]\right)
\end{aligned}$$

BitRev

A shuffle-elimination optimization trick. Decomposition is similar to walking a binary tree.

Parameter Generation in ML-KEM

Parameter Generation in ML-KEM

In practice, most objects (especially A) are too large for storage. Hence, the 256-bit "root seed" d is used with SHA3-512, SHAKE128, and SHAKE256 sponge hash functions in conjunction.

Parameters (Beginning)

$$(\underbrace{\rho}_{\text{256-bits}}, \underbrace{\sigma}_{\text{256-bits}}) = \text{SHA3-512}(d)$$

$$\hat{\mathbf{A}}[i,j] \leftarrow \text{RejectionSample}(\text{SHAKE128}(\rho||j||i))$$

$$\hat{\mathbf{s}} = \text{SamplePolyCBD}(\sigma)$$

Attacking ML-KEM

Learning With Error and Short Integer Solution

Consider the setup.

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$$

An attacker given $\mathbf{A}$ and $\mathbf{b}$ wants to solve for $\mathbf{s}$.
Brute-forcing $\mathbf{e}$ is infeasible ($\mathbf{e} \in R_q^3 \gtrapprox \{-2, -1, 0, 1, 2\}^{256\times 3}$)
They need to solve **short integer solution (SIS)** problem.

$$\underset{\mathbf{s}}{\operatorname{argmin}} ||\mathbf{A}\mathbf{s} - \mathbf{b}||$$

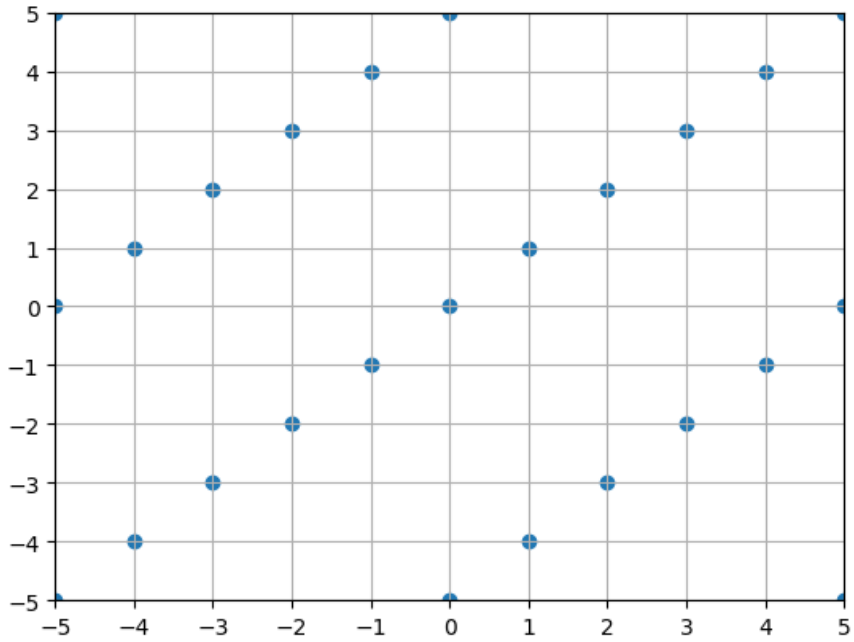
This is an NP-hard problem with no known quantum algorithm.
Best known algorithm is BKZ (with LLL as subroutine); 2^{182} operations for Kyber-768.

Related Problem: Closest Vector Problem

Closest Vector Problem (CVP): Example

$$\mathcal{B} = \left\{ \underbrace{\begin{pmatrix} 1 \\ 1 \end{pmatrix}}_{v_1}, \underbrace{\begin{pmatrix} -2 \\ 3 \end{pmatrix}}_{v_2} \right\}$$

For what c_1, c_2 is $c_1 v_1 + c_2 v_2$ closest to the origin, but not the origin?



CVP: Example

The solution here is $(1, 1), (-1, -1)$, but what if the dimension is 2304?

What if the entire lattice was shifted by some error?

Example: MSS_ADVANCE Revenge (picoCTF 2026)

Given secret polynomial $p(x) = c_0 + c_1 x + \cdots + c_{29} x^{29} \in \mathbb{Z}_{2^{32}}[x]$ where c_{29} is secret seed, and $\text{SHA256}(c_i) = c_{i-1}$ and p is a 1024-bit (309 digits) prime number that is given.
Given $(x_1, p(x_1) \mod p), (x_2, p(x_2) \mod p), \cdots, (x_{20}, p(x_{20}) \mod p)$, find c_{29}.
Note: 30 variables, 20 equations.

Caveat

Caveats

- Decryption Failure
  - ML-KEM has non-zero probability of failure.
  - For Kyber-768, once every 2^{164} operations.
- Key and Memory Size
  - Public key: 1184 bytes
- NTT Implementation Complexity
  - Difficult to prevent against side-channel.

Warning

- There is no guarantee that ML-KEM is quantum-safe.
  - We have not discovered an algorithm (classical or quantum) that efficiently attacks lattice problems.
- Not yet battle-tested for long-term

TLS 1.3

X25519MLKEM768

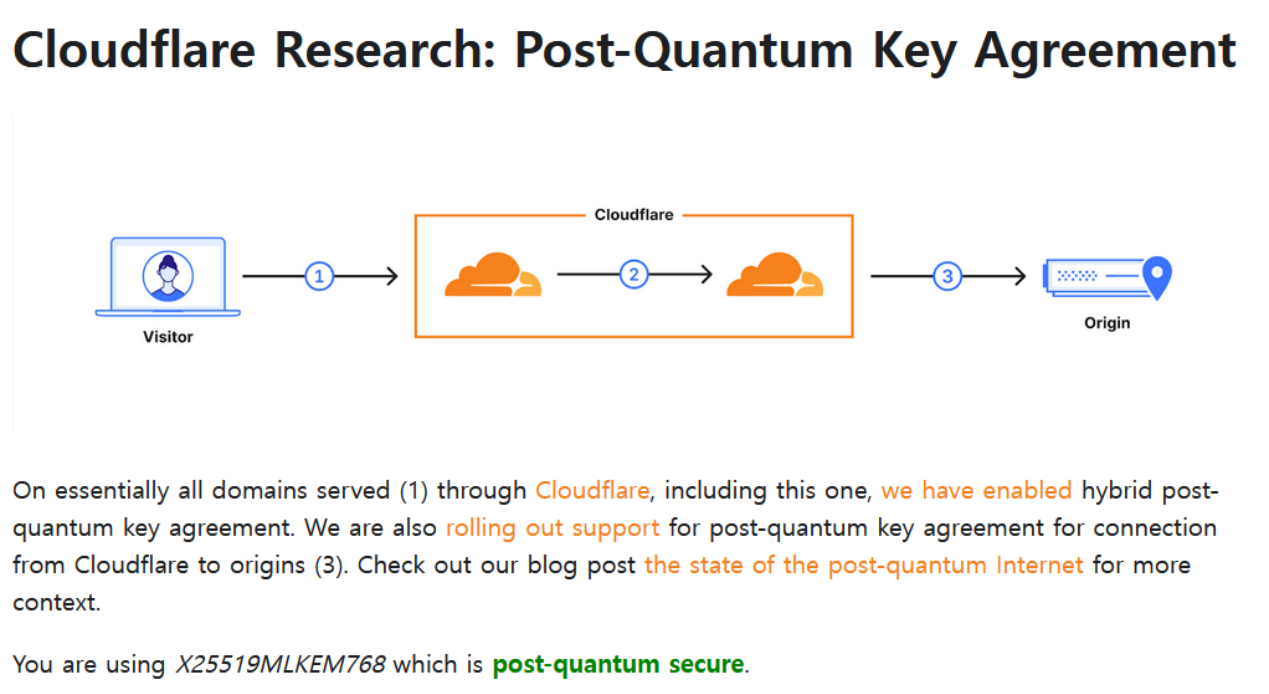


Figure: Cloudflare Research: Post-Quantum Key Agreement page shows that modern browsers support TLS 1.3